1. Individual portfolio + career research

Employer Postings & Research Analysis

Posting 1: Financial Consultant I
* **Employer & Title:** MetLife - Financial Consultant I
* **URL:** https://www.linkedin.com/jobs/search-results/?currentJobId=4482034096&eBP=NOT_ELIGIBLE_FOR_CHARGING&refId=hn8EBSwCUSgiQcVXF%2BGAnA%3D%3D&trackingId=bPM8%2FLO9eNZ1IpmUfpIcEg%3D%3D&keywords=financial%20analyst&origin=JOB_SEARCH_PAGE_LOCATION_AUTOCOMPLETE&geoId=101651951&f_SAL=f_SA_id_227001%3A276001
* **Access Date:** November 9, 2026
* **Eligibility & Location:** Legally authorized to work in the US | Hybrid (Warren, NJ)
* **Required Qualifications:** 1–3 years of related experience; High School Diploma (required); proficiency in Microsoft Office; strong accounting, analytical, and communication skills; ability to leverage AI tools for daily task efficiency.
* **Preferred Qualifications:** Bachelor's Degree or higher; Advanced Excel skills (Pivot Tables, VLOOKUPs, Macros); general understanding of the U.S. economy and employee benefits industry.
* **Core Responsibilities:**
  1. Serve as primary financial resource for National Accounts clients, overseeing financial reports, trend analysis, year-end accounting, 5500 reporting, and monthly revenue projections.
  2. Manage client billing, bank reconciliation, and provide financial support for RFPs, new implementations, and product additions.
  3. Partner with Account Executives and Client Services teams for renewals, underwriting support, client audits, and strategic planning.
* **Existing Evidence:** Deliverable 2 SQL/Python data analytics experience, automated workflow background, and financial report generation.
* **Skill / Experience Gap:** Group benefits 5500 reporting, specialized insurance accounting, and client billing/reconciliation workflows
* **Dated Next Action:** Complete advanced Excel financial modeling (Macros/Pivot Tables) and Group Benefits accounting review by November 19, 2026.

Posting 2: Sales Planning Analyst (Target Role)
* **Employer & Title:** Gellert Global Group — Sales Planning Analyst
* **URL:** https://www.linkedin.com/jobs/search-results/?currentJobId=4481154384&eBP=NOT_ELIGIBLE_FOR_CHARGING&refId=rmWStQg0%2BIZ1wg0CyR1qXA%3D%3D&trackingId=fMXoYM33Pe0w01cMWx5Qsw%3D%3D&keywords=fp%26a&origin=SEMANTIC_SEARCH_LANDING_PAGE&geoId=101651951&distance=-1.0&f_SAL=f_SA_id_227001%3A276001
* **Access Date:** November 9, 2026
* **Eligibility & Location:** Authorized to work in the US | Hybrid (Elizabeth, NJ — relocating to Madison, NJ in Q1 2027)

* **Required Qualifications:** Bachelor's degree in Finance, Mathematics, Economics, Business, or a related field; 1–2 years of related experience; strong analytical and problem-solving skills; advanced proficiency in Microsoft Excel.
* **Preferred Qualifications:** Food industry experience; familiarity with ERP and CRM tools such as Infor M3, Salesforce, or Exceedra.
* **Core Responsibilities:**
  1. Develop and evaluate cost models for imported finished goods, supporting Sales with cost estimates and profitability/margin analysis.
  2. Maintain costing data within Infor M3 ERP and Salesforce, resolving costing discrepancies and ensuring timely, accurate deliverables.
  3. Prepare financial sales reports on key performance metrics and build improved analytical tools, templates, and systems to streamline processes.
* **Existing Evidence:** Deliverable 2 quantitative data modeling, SQL/Python analytics, and automated spreadsheet/report generation.
* **Skill / Experience Gap:** Infor M3 ERP system workflows, food industry finished-goods cost modeling, and trade promotion management tools (Exceedra).
* **Dated Next Action:** Build a finished-goods import cost modeling template in Excel and review Infor M3 ERP costing architecture by November 19, 2026.

Posting 3: Finance Rotational Associate (Alternative Direction)
* **Employer & Title:** GAF - Finance Rotational Associate
* **URL:**
https://www.linkedin.com/jobs/search-results/?currentJobId=4482254104&eBP=CwEAAAGhwo9uHxZqx68jMAHhHWaH4R2egdeQZHvAcRTcP5wl5C48Ez1U_uyF4nwOHO5dIofrdWaynUyd6MGcD84Kk8qZDyNKZUTRKGqhpVc_d7OtZYW7RGIt_X1B1OaQeiPA5aELJd0OmN5ZFie6dmMvrRKGmPmDeMJvom2hNQPPHwj3EqcRX71l4OV7NFeg9uSDH1KbvgFMPM5jdFTu57FeDB4hEv-1H4kEgzWW4UBoWPNimWFVAzZHUnPD_-oEahVNetHflj3tEgWADpXHq3btKyit3fC99BRn0_3gf26tvKzuAQzBamFkJeoXaMdmthmPTYJMcSUB9Bgw5fP_hzW_K6k4P83SCStI_NK_3rK5B7tH-TxKLnui7qG4-EsW-Sjk6arbdof2jWYT0uRD5v76ZohW4PlWGpcmQ1KHYwZlt9rNJ1VGyBRVAbGFGVb_FzhL_RyDUHT6i5kFGaL59PZJDKX4OjzJXrWqE9Cb7lblw-4&refId=rmWStQg0%2BIZ1wg0CyR1qXA%3D%3D&trackingId=3DRmQ1qVGju%2BV40ba6vdFw%3D%3D&keywords=fp%26a&origin=SEMANTIC_SEARCH_LANDING_PAGE&geoId=101651951&distance=-1.0&f_SAL=f_SA_id_227001%3A276001
* **Access Date:** November 9, 2026
* **Eligibility & Location:** Eligible to work in the US without visa sponsorship | Hybrid (Parsippany, NJ)
* **Required Qualifications:** Bachelor's Degree in Accounting; intermediate proficiency in Microsoft Excel and Word; strong foundational knowledge of accounting concepts, practices, and procedures.
* **Preferred Qualifications:** CPA or CPA candidate; critical thinking, problem-solving, and analytical skills; previous accounting/finance internship or co-op experience; familiarity with PeopleSoft, G-Suite, IBM Cognos TM1, or SAP Analytics Cloud (SAC).
* **Core Responsibilities:**

1. Execute Corporate Accounting monthly general ledger close, ASC 842 lease transaction reconciliations, asset management counts, and year-end audit support.
2. Perform Revenue Management financial forecasting, pricing action analysis (price/volume tradeoffs, P&L analysis, customer segmentation), and profitability reporting using Excel, TM1, and SAC.
3. Partner cross-functionally with Sales, FP&A, IT, and Product Management to build pricing blueprints, playbooks, and continuous process improvements.

* **Existing Evidence:** Quantitative data modeling, SQL/Python analytics, and automated spreadsheet/financial report generation.
* **Skill / Experience Gap:** ASC 842 lease accounting standards, enterprise ERP/CPM tools (PeopleSoft, TM1, SAC), and revenue/pricing management playbooks.
* **Dated Next Action:** Review ASC 842 lease accounting principles and practice multi-dimensional financial forecasting in Excel by November 19, 2026.

Direction Comparison & Provisional Target Selection

* Career Directions Compared: Commercial Sales & Cost Analysis (Gellert Global Group) vs. Financial Client Operations (MetLife) vs. Corporate Accounting & Revenue Management (GAF Rotational).
* **Sales Planning Analyst**: Focuses on landed cost modeling, margin evaluation, ERP pricing workflows, and cross-functional sales/finance strategy.
* **Financial Consultant I:** Focuses on client account administration, benefits billing, 5500 tax reporting, and operational bank reconciliations.
* ***Finance Rotational Associate:*** Focuses on ASC 842 lease compliance, GL monthly close accounting, and enterprise TM1/SAC revenue analytics.
* **Provisional Target Role:** Sales Planning Analyst (Gellert Global Group).
* **Target Reader Definition:** Commercial Finance Manager, FP&A Director, or Senior Business Analyst who values automated data pipelines, landed-cost accuracy, granular margin analysis, and reliable SQL/Python integration over static manual spreadsheets.
* **Desired Reader Outcome:** The reader concludes that I possess strong quantitative modeling capabilities, master landed-cost and margin structures, automate analytical tools via SQL/Python/Excel, and seamlessly translate complex cost data into actionable commercial insights.

Responsibility-to-Evidence Mapping

1. **Employer Responsibility:** "Develop and evaluate cost models for imported finished goods, supporting Sales with cost estimates and profitability/margin analysis."

  * **Portfolio Evidence:** Deliverable 2 Quantitative Data Modeling Case Study featuring multi-variable landed-cost algorithms, dynamic margin sensitivity models, and margin threshold rules.
2. **Employer Responsibility:** "Prepare financial sales reports on key performance metrics and build improved analytical tools, templates, and systems to streamline processes."
  * **Portfolio Evidence:** Deliverable 2 Automated Analytics Pipeline detailing custom SQL extraction scripts, Python data transformation workflows, and automated Excel reporting templates.
3. **Employer Responsibility:** "Maintain costing data within Infor M3 ERP and Salesforce, resolving costing discrepancies and ensuring timely, accurate deliverables."
  * **Portfolio Evidence:** Deliverable 3 Collaborative System Project highlighting schema design, structured database record management, data validation logic, and team git revision tracking.

Layout Comparison & Selection

* **Layout Option A (Single-Column Sequential Feed):**
  * **Structure:** A linear vertical layout presenting background narratives, formula listings, spreadsheet snips, and static output tables sequentially.
  * **Pros/Cons:**Mobile-friendly reading, but hides high-impact metric tables and landed-cost formulas under lengthy text blocks, making quick evaluation difficult for financial hiring managers.
* **Layout Option B (Two-Column Split Workspace - SELECTED):**
  * **Structure:** A side-by-side workspace displaying core financial logic, margin sensitivity assumptions, and structural cost drivers on the left panel, with interactive pricing templates, SQL query snippets, and dynamic P&L charts on the right panel. High visual clarity and density; enables hiring managers to review quantitative formulas, database logic, and analytical visual outputs simultaneously without endless vertical scrolling.
  * **Pros/Cons:** High visual clarity and density; enables hiring managers to review quantitative formulas, database logic, and analytical visual outputs simultaneously without endless vertical scrolling.
* **Selection Reason:** Option B directly serves the Target Reader by presenting concrete quantitative proofs (SQL scripts, landed-cost formulas, and interactive dashboards) alongside strategic context immediately upon page render.

Site Map & Traceability Matrix

* GET / — Professional Bio, Target Role Alignment, Resume Download, GitHub/LinkedIn Links, and Contact Route [PLANNED]
* GET /projects/cost-modeling — Deliverable 2 Landed Cost & Profitability Analysis Case Study with Interactive Scenario Calculator [LOCAL SETUP]

* GET /projects/group-data-system — Deliverable 3 Collaborative Database & Automated Analytics Pipeline Case Study [PLANNED]
* POST /api/v1/cost-calculate — Live API Endpoint for Dynamic Import Costing & Margin Evaluation [LOCAL SETUP]

---

2. Individual RAG app + technical research

Project: NJIT Academic Assistant
Current Stage: Planning
Project Type: Individual RAG Application

1. Project Overview and Problem

For my individual project, I plan to create an AI-powered academic assistant that helps NJIT students find information about university policies, courses, graduation requirements, and academic resources.

Students sometimes need to search through different websites and long documents to find answers to simple questions. My application will allow them to ask questions in natural language and receive answers supported by official university documents.

For example, a student could ask, "What happens if I withdraw from a class?" The application would search the available documents, find relevant information, and provide an answer with a citation.

My goal is to make academic information easier to access while reducing the risk of incorrect AI-generated answers.

2. Research: RAG, Vector and Graph Databases

Retrieval-Augmented Generation (RAG)

RAG is a technique that improves how language models answer questions by retrieving information from external documents. Instead of relying only on what the language model already knows, the system searches a collection of documents and uses relevant information to generate a response.

A basic RAG system follows these steps:

1. Import documents.
2. Divide documents into smaller text chunks.
3. Convert the chunks into embeddings.
4. Store the embeddings in a vector database.
5. Retrieve relevant chunks when a question is asked.
6. Generate an answer using those chunks.

I chose RAG because my application needs to answer questions based on official NJIT information rather than general AI knowledge.

Research source: LangChain RAG documentation — https://docs.langchain.com/oss/python/langchain/rag

Vector Databases

Vector databases store and search embeddings, which are numerical representations of information. They allow an application to find text with similar meanings even when the exact words are different.

I researched ChromaDB and FAISS.

ChromaDB supports embeddings, documents, and metadata, making it useful for a document-based RAG application.

FAISS is another option that provides efficient vector similarity search. However, it is mainly a similarity-search library, so additional document and metadata management would be needed for my application.

Research sources:

- ChromaDB: https://docs.trychroma.com/
- FAISS: https://github.com/facebookresearch/faiss

Graph Databases

Graph databases store information using nodes and relationships. Nodes represent entities such as courses, departments, and degree programs, while relationships describe how those entities are connected.

For example:

- Entity: MATH 213
- Entity: Mathematics Department
- Relationship: MATH 213 belongs to the Mathematics Department

Another possible relationship is a course requiring another course as a prerequisite. These relationships would need to be verified using the official course catalog.

I researched Neo4j, which supports graph databases and relationship-based queries.

Research source: https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/

3. Database Comparison and Final Choice

Option	Advantages	Limitations
ChromaDB	Simple Python integration, metadata support, local development	Does not provide full graph relationship traversal
FAISS	Efficient vector similarity search, suitable for local applications	Requires additional logic for document and metadata management
Neo4j	Supports entities, relationships, and connected queries	Requires more data modeling and graph queries
Hybrid Vector + Graph	Combines semantic search with relationship traversal	Greater complexity, development time, and maintenance

Final Choice: ChromaDB (Vector Database)

I chose ChromaDB because my project focuses mainly on retrieving information from university documents. Students will mostly ask questions about policies, deadlines, and academic requirements.

A vector database is a practical choice because it can retrieve relevant text using semantic similarity without requiring me to create and maintain an entire knowledge graph.

I considered using both a vector database and a graph database, but I decided that the additional complexity was unnecessary for the first version.

If I expand the application later, Neo4j could help with questions involving multiple connected courses, prerequisites, and degree requirements.

4. Chunking, Embeddings, Entities, Relationships, and Import

Chunking: I plan to divide documents into sections of approximately 1,000 characters with 200 characters of overlap. This should help preserve context between sections. I will compare this with a smaller chunk size during testing to determine which works better.

Embeddings: I plan to use a Sentence Transformers embedding model to convert document chunks and user questions into numerical vectors. Similar meanings should produce vectors that are close together in the embedding space.

Entities: Important entities in my documents may include course names, academic departments, degree programs, credit requirements, and university policies.

Relationships: Relationships may include a course belonging to a program, a course requiring a prerequisite, or a policy applying to a specific student category. For the first version, I will mainly use this information as searchable text and metadata rather than build a graph database.

Import Process: I plan to collect a small set of permitted NJIT documents, extract their text, remove unnecessary formatting, split them into chunks, create embeddings, and store them in ChromaDB.

Each chunk will include metadata such as:

- Document title
- Original source URL
- Page or section number, if available
- Catalog year or publication date, if available
- Unique chunk ID

This metadata will help the application provide traceable sources.

5. Permitted Data Sources

I plan to use publicly accessible documents and webpages from official NJIT sources, subject to the university's website terms and any applicable reuse restrictions.

My initial sources will include:

1. NJIT Undergraduate Catalog: https://catalog.njit.edu/
2. NJIT Office of the Registrar: https://www.njit.edu/registrar/
3. Official academic policy pages
4. Public course descriptions and degree requirements

Before importing a document, I will confirm that it is publicly accessible and appropriate for my project. I will avoid collecting private student records, grades, personal information, and restricted documents.

I will begin with approximately 10–20 pages or documents to keep the project manageable.

I will also track the source and catalog year because university policies can change.

6. Proposed Application Architecture

The application will have two main processes.

Document Import and Indexing:

Official NJIT Documents → Text Extraction → Chunking → Embedding Model → ChromaDB

Question Answering:

Student Question → Question Embedding → ChromaDB Similarity Search → Relevant Text Chunks → Language Model → Answer With Citations

I plan to develop the application using:

Component	Technology
Programming language	Python
Vector database	ChromaDB
RAG framework	LangChain
Embedding model	Sentence Transformers
Language model	API-based LLM, depending on available access and cost
User interface	Streamlit
Version control	GitHub
Initial hosting	Local computer

The system will retrieve approximately three to five relevant chunks for each question. This number may change depending on evaluation results.

The language model will receive instructions to answer using the retrieved information, include citations, and acknowledge when the available documents do not support an answer.

7. Research Checks and Decision Evidence

I reviewed the technical documentation to check whether the tools I selected fit the project requirements.

Research Check	Evidence From Documentation	Decision
Can ChromaDB store documents and metadata?	Chroma documentation describes collections that store embeddings, documents, and metadata	ChromaDB fits the project
Can ChromaDB preserve information locally?	Chroma documents a PersistentClient for local disk storage	Use persistent storage during development
Can a graph database represent course relationships?	Neo4j documentation describes nodes and typed relationships	Graph databases are useful but optional
Does LangChain support the RAG workflow?	LangChain documents loading, splitting, embedding, retrieving, and generating answers	Use LangChain for the RAG pipeline
Can Streamlit host an application?	Streamlit Community Cloud documentation describes GitHub-based deployment and resource limits	Consider cloud deployment after local testing

These are documentation-based research checks. They confirm that the technologies support the proposed design, but they do not prove that my implementation is working yet.

Sources:

- https://docs.trychroma.com/
- https://neo4j.com/docs/getting-started/
- https://docs.langchain.com/oss/python/langchain/rag
- https://docs.streamlit.io/deploy/streamlit-community-cloud

8. Evaluation Questions and Testing Plan

I will create a small evaluation dataset containing at least 20 questions and expected answers verified against the imported documents.

Example evaluation questions:

1. What is the minimum GPA required for undergraduate graduation?
2. What is considered full-time undergraduate enrollment?
3. What happens when a student withdraws from a course?
4. What are the requirements for the Dean's List?

5. Where can students find academic support services?
6. Can a student repeat a course?
7. How are credit requirements described in a degree program?
8. What should the assistant do if information is missing?

I will check the following:

Retrieval Accuracy: Whether the correct source document appears among the top retrieved chunks.

Answer Correctness: Whether the generated response matches the information in the official document.

Citation Accuracy: Whether the citation actually supports the answer.

Unsupported Questions: Whether the application admits that information is unavailable instead of inventing an answer.

Response Time: How long the application takes to answer a question.

My proposed success targets are:

- At least 80% of evaluation questions receive correct, source-supported answers.
- At least 90% of test questions retrieve relevant source information among the top five chunks, when the answer is present.
- All factual answers include a source reference.
- Unsupported questions produce an appropriate explanation rather than an invented policy.

These are targets for future evaluation, not results I have already achieved.

9. Five Observable User Stories and Acceptance Checks

User Story 1: Ask Questions

As an NJIT student, I want to ask questions about academic policies so I can find information without reading multiple documents.

Acceptance Check: Enter a question about a policy included in the dataset. Verify that the application returns a correct, relevant response.

Evidence to collect: Screenshot of the question, answer, and matching source text.

User Story 2: View Sources

As a student, I want to see the source of an answer so that I can verify the information.

Acceptance Check: Ask a supported question and confirm that the response includes a citation linking to the appropriate document.

Evidence to collect: Screenshot showing a working source link and the supporting passage.

User Story 3: Natural Language Search

As a student, I want to ask questions using different wording without needing to know exact keywords.

Acceptance Check: Ask two questions with similar meanings but different wording and compare the retrieved results.

Evidence to collect: A comparison of both questions and the retrieved documents.

User Story 4: Handle Missing Information

As a student, I want the assistant to admit when information is unavailable so that I do not receive misleading answers.

Acceptance Check: Ask a question whose answer is not supported by the imported documents and verify that the system does not invent an answer.

Evidence to collect: Screenshot of the unsupported question and the application's response.

User Story 5: Update Documents

As the developer, I want to import updated documents so that the assistant can use newer information.

Acceptance Check: Replace a test document with a revised version, update the vector index, and verify that the application retrieves the new information instead of the outdated version.

Evidence to collect: Import logs, before-and-after responses, and relevant database records.

10. Hosting, Resources, and Constraints

Because this is an individual project, I plan to prioritize a working local application before attempting cloud deployment.

I will use Python and a local ChromaDB database to avoid requiring a separate database server during early development.

Potential limitations include:

- Limited memory and computing resources
- Language model API costs and request limits
- Incorrect or incomplete PDF text extraction
- Documents containing outdated academic policies
- Retrieval errors caused by chunk sizes or ambiguous questions
- The possibility of hallucinated or unsupported responses
- Limited time for developing and evaluating a more complex application

Streamlit Community Cloud is one possible deployment option, but its resource limits and nonpersistent local storage behavior may require me to use externally persisted data or rebuild the vector index during deployment.

I will keep the first version small, use public documents, and evaluate the application locally before expanding.

11. Planned Improvements and Peer Feedback

During the Improve stage, I plan to ask a peer to test the application without my assistance.

I will ask the peer to perform common tasks, including finding a university policy, checking a citation, and asking a question that is not answered in the documents.

I will record:

1. The name or identifier of the peer reviewer
2. The date of the test
3. The question or feature tested
4. The issue the reviewer found
5. The specific change I made
6. The result of retesting the same question

For example, if a peer finds that a citation points to the wrong section, I will investigate the document metadata, modify the citation logic, and retest the question.

I will include screenshots or test logs documenting the original problem, the code change, and the improved result.

This peer-testing activity is planned for the Improve stage and has not yet been completed.

12. Understanding, Limitations, and Individual Contribution

This project will help me understand how RAG combines document processing, semantic search, vector databases, and language models.

One important concept I learned during planning is that a RAG system does not automatically guarantee correct answers. The system can still retrieve irrelevant information, miss important context, or generate an answer that is not fully supported by the documents.

I also learned that vector databases and graph databases solve different problems. Vector databases are useful for finding similar information, while graph databases are useful for representing and querying connected entities.

I chose a vector database because it better matches the first version of my application and is more manageable for an individual project.

My Individual Contribution: I am responsible for selecting the project idea, evaluating database alternatives, defining the application architecture, identifying permitted data sources, creating evaluation questions, implementing the application, testing the results, and documenting improvements.

Help and Resources Used: I used technical documentation to research RAG architectures, databases, and hosting options. I also used ChatGPT to help organize the planning document, compare technologies, and develop test ideas. I will independently verify the technical details and will be responsible for implementing and testing the application.

Current Limitations: My project is currently in the planning phase. I have not yet completed the application, measured retrieval accuracy, performed implementation tests, or collected peer feedback. Those activities will be documented as the project progresses.

13. Expected Outcome

By the end of this project, I aim to have a functional academic assistant that answers questions based on permitted NJIT documents, retrieves relevant information using a vector database, and provides source citations.

I want the finished application to demonstrate not only how RAG works but also how the accuracy and reliability of an AI system can be evaluated.

I will measure the application's performance using a documented set of evaluation questions, save evidence of successful and unsuccessful tests, and use peer feedback to make at least one verified improvement.

My goal is to create a useful, manageable application while demonstrating my understanding of the technologies, design decisions, limitations, and testing process.